Text as Data

W6: Human Coding and Document Classification

Yen-Chieh Liao 廖彥傑

2026-10-16

Fall 2026 (115-1) · National Taiwan University (GIND)
NTLDEV 7191 · Fri. 16:30–18:20 · 國發 204
Last compiled: 2026-10-16 12:35:30 (Asia/Taipei)

Learning objectives: from W4 to W6

Task	W4: dictionaries	W6: classification
Build a measure	Select words and rules	Learn from labelled comments
Use human evidence	Validate dictionary output	Train and evaluate classifiers
Develop the method	Inspect KWIC; revise rules	Fit ML algorithms and predict new texts
Assess results	Read errors and sensitivity	Reserved test and human benchmark

Practice: apply machine learning algorithms to two real datasets:

1. Political hate speech in Taiwan.

2. Grandstanding statements in US congressional hearings.

Course progression: W4: Dictionaries and Sentiment Analysis to W6.

Cases we use in W6

Research case	Source	Teaching connection
Wang, Day & Wu (2022)	IEEE Access	Hate speech: Chinese character counts
Gillion (2016)	*Governing with Words*	Coding rules and agreement
Benoit et al. (2016)	APSR	Crowd coding and reproducibility
Boydstun (2013)	*Making the News*	Codebooks and rare categories
Nielsen (2017)	*Deadly Clerics*	Found labels and validation
Park (2021)	JOP	Grandstanding: mlr3 linear SVM

Original-study sources accompany each literature case. Classification practice uses the data from Wang, Day & Wu (2022) and Park (2021).

Wang, Day & Wu: political hate speech in Taiwan

IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received October 25, 2021, accepted February 16, 2022, date of publication March 21, 2022, date of current version April 29, 2022.

Digital Object Identifier 10.1109/ACCESS.2022.3160712

Political Hate Speech Detection and Lexicon Building: A Study in Taiwan

CHIH-CHIEN WANG, MIN-YUH DAY, (Member, IEEE), AND CHUN-LIAN WU
Graduate Institute of Information Management, National Taipei University, New Taipei City 23741, Taiwan

Corresponding author: Chih-Chien Wang (wangson@mail.ntpu.edu.tw)

This work was supported in part by the Ministry of Science and Technology (MOST), Taiwan, under Grant 109-2410-H-305-029.

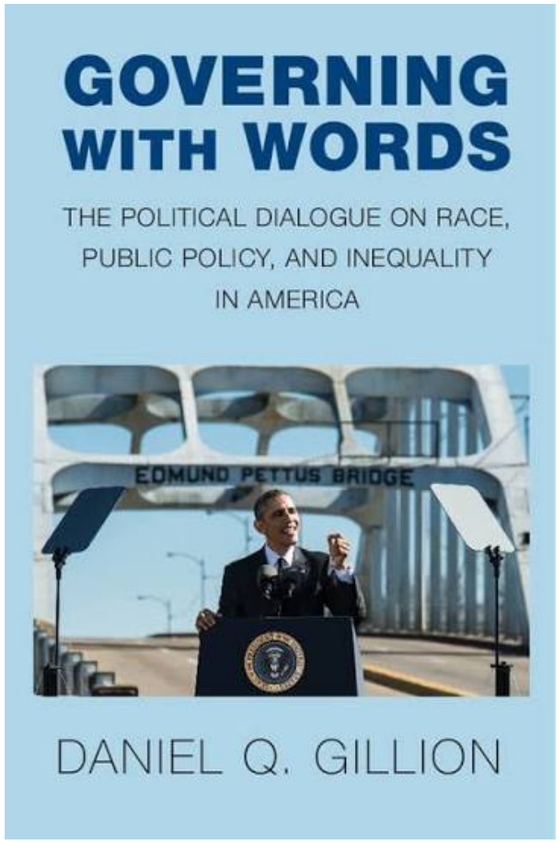
ABSTRACT There is the minimal restriction to users' speech in cyberspace. The Internet provides a space where people can freely present their speech, which puts a Utopian sense of freedom of speech into practice. However, the appearance of hate speech is a significant side effect of online freedom of speech. Some users use hate speech to attack others, making the attacked targets uncomfortable. The proliferation of hate speech poses severe challenges to cyber society. Users may hope that social media platforms and online communities promote anti-hate speech. However, hate speech detection is still a developing technology that requires system developers to create a method to detect unacceptable hate speech while maintaining the online freedom of speech environment. No excellence detection approach has yet been proposed, although some literature has focused on it. The current study proposes an approach to build a political hate

2022

- **Texts:** Chinese political news comments.
- **Study:** a hate-speech lexicon and BERT classifiers.
- **W6 practice:** NB on the Wang, Day & Wu (2022) data.

Wang, Day & Wu (2022).

Gillion: measuring political discussion of race



Cambridge University Press, 2016

- **Question:** how do politicians discuss racial and ethnic minority concerns?
- **Texts:** presidential public remarks and congressional oral remarks.
- **Application:** humans label paragraphs; classifiers extend the coding to the full corpus.
- **Research use:** compare political attention across politicians and time.

§17.1. Gillion (2016), Governing with Words, Appendix A, Appendix B.

Gillion: coding boundaries and agreement

Unit: code each paragraph separately, because one speech can cover several topics. Exclude written bills and procedural material.

Training labels: two assistants code random samples: 300 paragraphs per presidential administration and 1,000 per congressional session. Their decisions give the classifier examples of what counts as racial discussion.

Coding rule: require an explicit reference to a racial group, race-specific policy, minority figure, or race-specific issue. For example, school funding alone does not qualify, but racial inequality in school funding does.

Agreement: reported intercoder reliability is **97%**. Gillion resolves disagreements. This checks human consistency. Classifier accuracy needs its own test.

Discuss: Could this rule miss implicit racial appeals?

Gillion (2016), Appendix A, pp. 163–166. The 97% concerns coders, not classifier accuracy.

Gillion: interpreting the measure

- **Missed content:** coded racial messages without an explicit racial reference.
- **Example:** “inner city” can carry racial meaning in context; the phrase alone is not enough to infer a racial appeal.
- **Measurement lesson:** 97% agreement shows consistent coding. It does not establish that implicit appeals are captured.

Course interpretation of Gillion (2016), Appendix A · Racial cue example: White (2007).

Benoit et al.: crowd coding political text

American Political Science Review Vol. 110, No. 2 May 2016
doi:10.1017/S0003055416000058
© American Political Science Association 2016

Crowd-sourced Text Analysis: Reproducible and Agile Production of Political Data

KENNETH BENOIT *London School of Economics and Trinity College*
DREW CONWAY *New York University*
BENJAMIN E. LAUDERDALE *London School of Economics and Political Science*
MICHAEL LAVER *New York University*
SLAVA MIKHAYLOV *University College London*

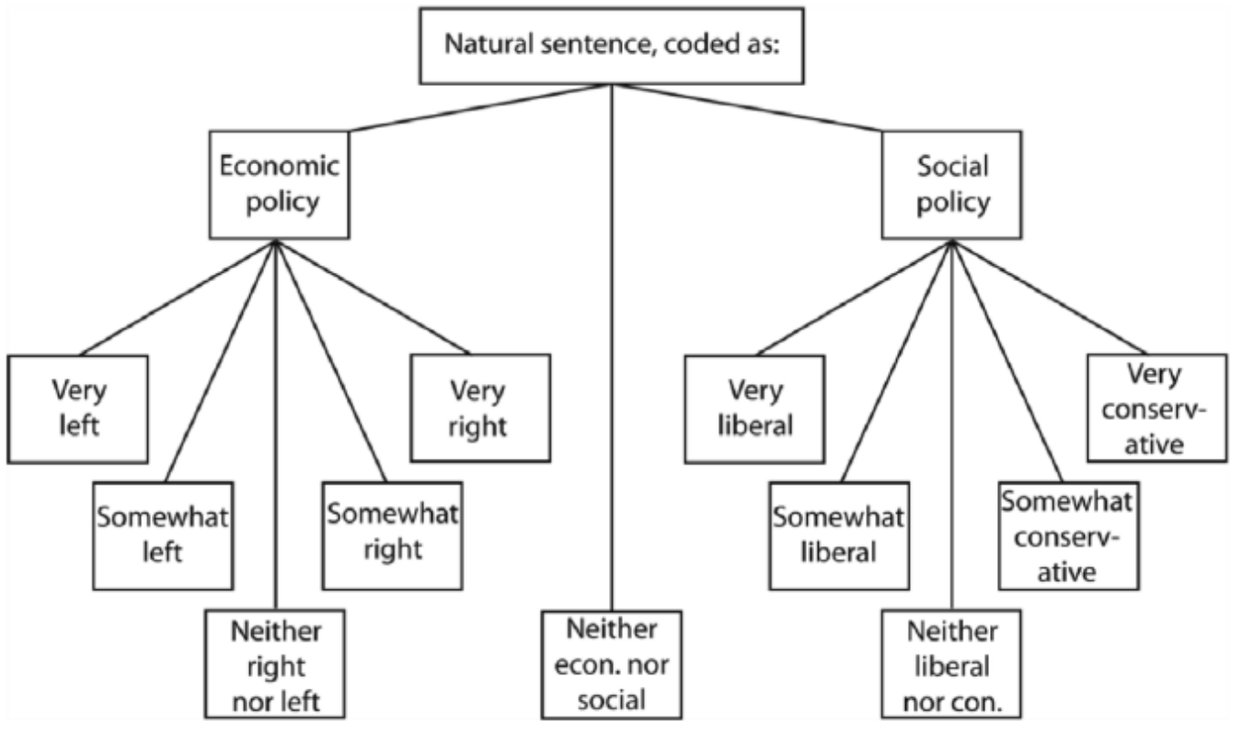
Empirical social science often relies on data that are not observed in the field, but are transformed into quantitative variables by expert researchers who analyze and interpret qualitative raw sources. While generally considered the most valid way to produce data, this expert-driven process is inherently difficult to replicate or to assess on grounds of reliability. Using crowd-sourcing to distribute text for reading and interpretation by massive numbers of nonexperts, we generate results comparable to those using experts to read and interpret the same texts, but do so far more quickly and flexibly. Crucially, the data we collect can be reproduced and extended transparently, making crowd-sourced datasets intrinsically reproducible. This focuses researchers' attention on the fundamental scientific objective of specifying reliable and replicable methods for collecting the data needed, rather than on the content of any particular dataset. We also show that our approach works straightforwardly with different types of political text, written in different languages. While findings reported here concern text analysis, they have far-reaching implications for expert-generated data in the social sciences.

APSR, 2016 · first-page excerpt

- **Texts:** manifesto sentences.
- **Coding:** repeated crowd judgments.
- **Validation:** compare with experts.

Crowd coding: a simple coding scheme

FIGURE 1. Hierarchical Coding Scheme for Two Policy Domains with Ordinal Positioning



- **First:** classify the topic.
- **Then:** rate the position.
- **Design:** keep tasks simple.

Figure 1 · original coding scheme

Benoit et al. (2016), APSR · Figure 1, p. 281.

Crowd coding: reproducing the judgments

TABLE 4. Comparison Results for Replication of Immigration Policy Crowd Coding

	Wave		
	Initial	Replication	Combined
Total Crowd Codings	24,674	24,551	49,225
Number of Coders	51	48	85
Total Sentences Coded as Immigration	280	264	283
Correlation with Benoit Expert Survey (2010)	0.96	0.94	0.96
Correlation with CHES 2010	0.94	0.91	0.94
Correlation of Results between Waves			0.93

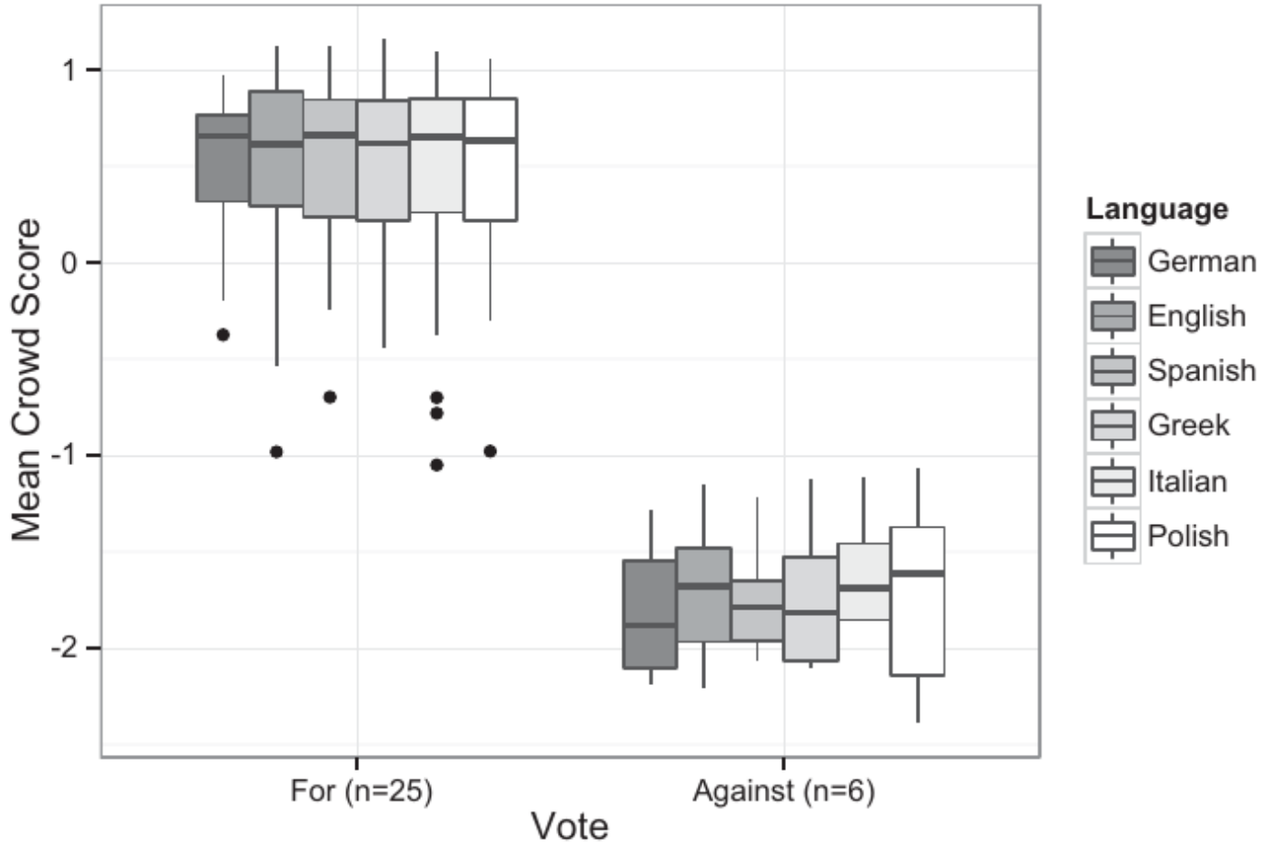
Table 4 · immigration coding replication

- **Repeat:** two coding waves.
- **Result:** party scores correlate **.93**.
- **Scope:** aggregate reproducibility.

Benoit et al. (2016), APSR · Table 4, p. 291.

Crowd coding: the coal-mine policy case

FIGURE 7. Scored Speeches from a Debate over State Subsidies by Vote, from Separate Crowd-sourced Text Analysis in Six Languages



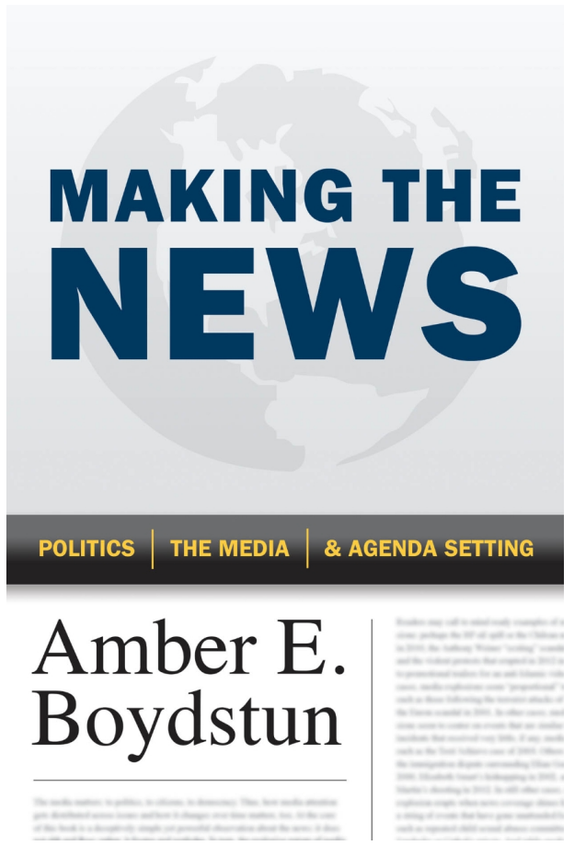
Note: Aggregate scores are standardized for direct comparison.

- **Texts:** 36 EP speeches.
- **Plot:** 31 votes; six languages.
- **Check:** scores against votes.

Figure 7 · coal-mine aid debate

Benoit et al. (2016), APSR · Figure 7, p. 292.

Boydstun: measuring the news agenda

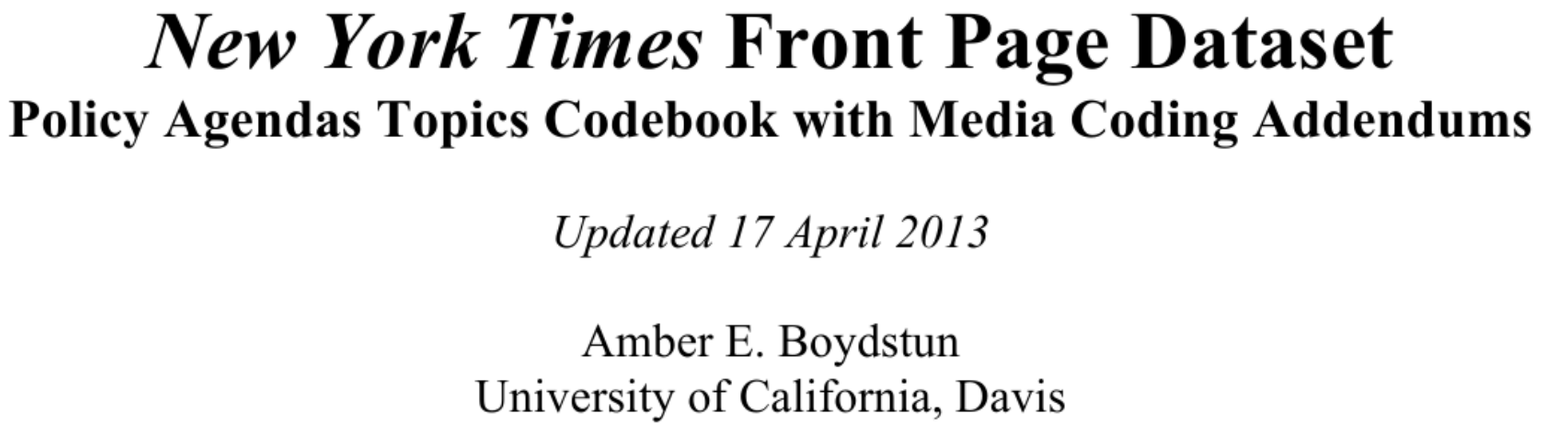


Book cover · University of Chicago Press, 2013

- **Question:** how is news attention distributed across policy issues over time?
- **Data:** New York Times front-page stories, 1996–2006.
- **Application:** human coders assign policy topics; counts track media attention across issues and time.
- **W6 connection:** construct consistent labels before comparing issues.

§18.2.7. Boydstun (2013), book supplements.

Boydstun: a codebook for newspaper topics



***New York Times* Front Page Dataset**

Policy Agendas Topics Codebook with Media Coding Addendums

Updated 17 April 2013

Amber E. Boydstun
University of California, Davis

Original coding manual · 2013

- **Evidence:** headline and first three paragraphs.
- **Label:** one primary topic and a summary.
- **Consistency:** align storyline labels; clarify unclear cases.
- **Lesson:** define the evidence coders may use.

Book connection: §18.2.7. Boydstun (2013), media codebook, pp. 1–5.

Boydstun: labels reveal unequal attention

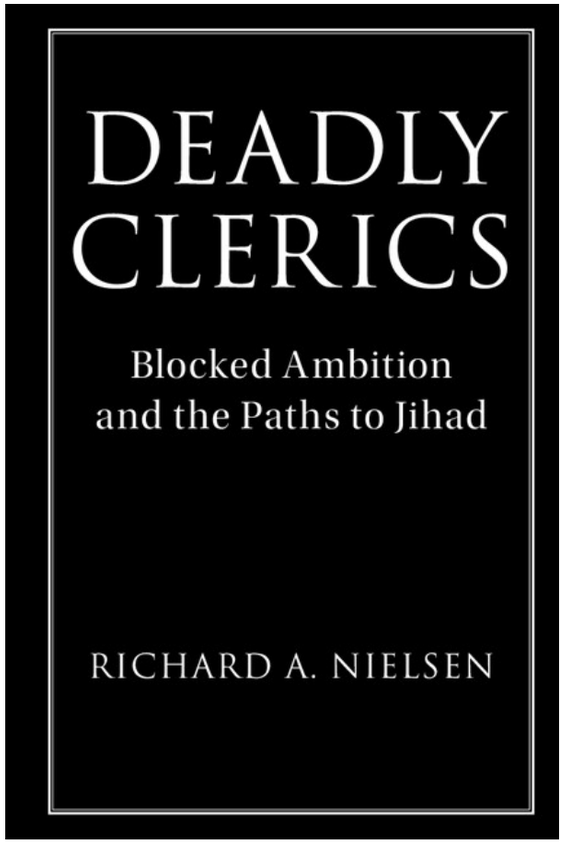
Table A.1. *Times* front-page attention by topic, 1996–2006, with monthly summary statistics

Topic	Stories	Share of all stories
International affairs and foreign aid	6,354	20.5%
Defense	4,479	14.4%
Health	1,799	5.8%
Energy	299	1.0%

Selected rows; full corpus: **31,034 stories**.

Boydstun (2013), Table A.1. Selected topic counts from the original study.

Nielsen: estimating ideological language

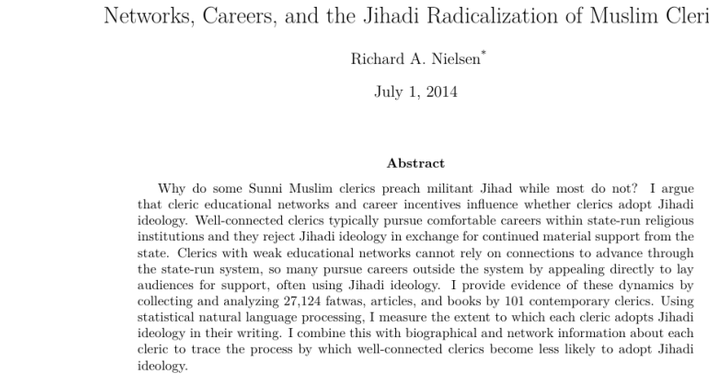


Cambridge University Press, 2017

- **Texts:** Arabic writings by clerics.
- **Measure:** language-based jihad score.
- **Use:** study networks, careers and ideological differences.

§19.3. Nielsen (2017), Deadly Clerics.

Nielsen: from reference texts to a score



Networks, Careers, and the Jihadi Radicalization of Muslim Clerics

Richard A. Nielsen*

July 1, 2014

Abstract

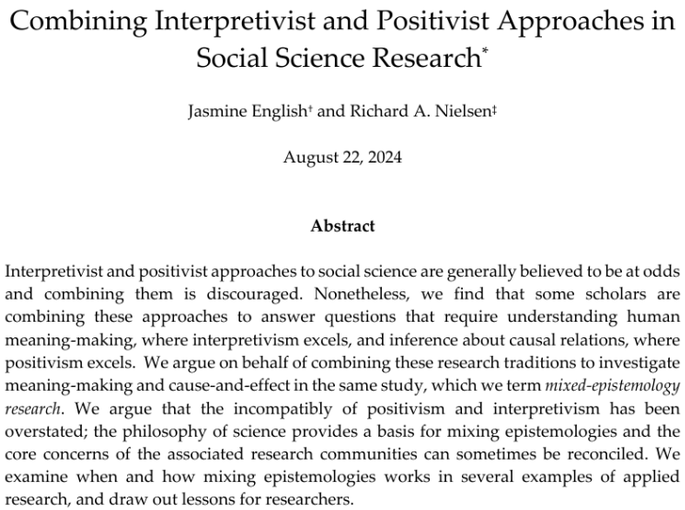
Why do some Sunni Muslim clerics preach militant Jihad while most do not? I argue that cleric educational networks and career incentives influence whether clerics adopt Jihadi ideology. Well-connected clerics typically pursue comfortable careers within state-run religious institutions and they reject Jihadi ideology in exchange for continued material support from the state. Clerics with weak educational networks cannot rely on connections to advance through the state-run system, so many pursue careers outside the system by appealing directly to lay audiences for support, often using Jihadi ideology. I provide evidence of these dynamics by collecting and analyzing 27,124 fatwas, articles, and books by 101 contemporary clerics. Using statistical natural language processing, I measure the extent to which each cleric adopts Jihadi ideology in their writing. I combine this with biographical and network information about each cleric to trace the process by which well-connected clerics become less likely to adopt Jihadi ideology.

Working paper · July 2014

- **Measure:** how strongly a cleric's writing expresses militant Jihadi ideology.
- **Supervision:** 765 jihadist-curated texts versus 1,951 sampled cleric texts, a mixed baseline.
- **NB weights:** words more common in jihadist references raise the score; words more common in the baseline lower it.
- **Aggregation:** join each cleric's writings and add these word weights.
- **Interpretation:** higher scores mean language more like the jihadist references; the measure is continuous.

Method supplement: Nielsen (2014), §4.1, pp. 13–14. Earlier working paper, separate from the 2017 book.

Nielsen: independent checks on the scores



Combining Interpretivist and Positivist Approaches in Social Science Research*

Jasmine English† and Richard A. Nielsen‡

August 22, 2024

Abstract

Interpretivist and positivist approaches to social science are generally believed to be at odds and combining them is discouraged. Nonetheless, we find that some scholars are combining these approaches to answer questions that require understanding human meaning-making, where interpretivism excels, and inference about causal relations, where positivism excels. We argue on behalf of combining these research traditions to investigate meaning-making and cause-and-effect in the same study, which we term *mixed-epistemology research*. We argue that the incompatibly of positivism and interpretivism has been overstated; the philosophy of science provides a basis for mixing epistemologies and the core concerns of the associated research communities can sometimes be reconciled. We examine when and how mixing epistemologies works in several examples of applied research, and draw out lessons for researchers.

- **Paper's argument:** combine understanding actors' meanings with quantitative tests of causes.
- **Nielsen case:** treat clerics as scholars, then test whether blocked careers predict jihadist rhetoric.
- **Language check:** explain why "apostasy" signals jihadist ideology in context.
- **External checks:** compare scores with biographies and scholarly classifications (2014), plus a jihadist infographic criticizing non-jihadist preachers (2017).
- **W6 lesson:** validate what the score means using evidence outside the training texts.

Book connection: §20.3. Nielsen (2017), Ch. 5. Nielsen (2014), pp. 15–17; English & Nielsen (2024), pp. 23–24.

Nielsen: the limits of external validation

A missing biographical label is not a confirmed negative.

Some biographies identify jihadist commitments explicitly. Others omit ideology. Silence cannot establish non-jihadist status.

Course discussion: could a text criticizing jihadism receive a high score because it discusses the same vocabulary?

Check passages in context and inspect differences across genres and authors. Independent benchmarks and close reading answer complementary questions.

For our hate-speech task, inspect comments quoting abuse in order to reject it.

Nielsen (2013), dissertation, §4.4.1, pp. 121–122; Nielsen (2014), p. 17 n. 16. Discussion and hate-speech application are course extensions.

Park: measuring political grandstanding

When Do Politicians Grandstand? Measuring Message Politics in Committee Hearings

Ju Yeon Park, University of Essex

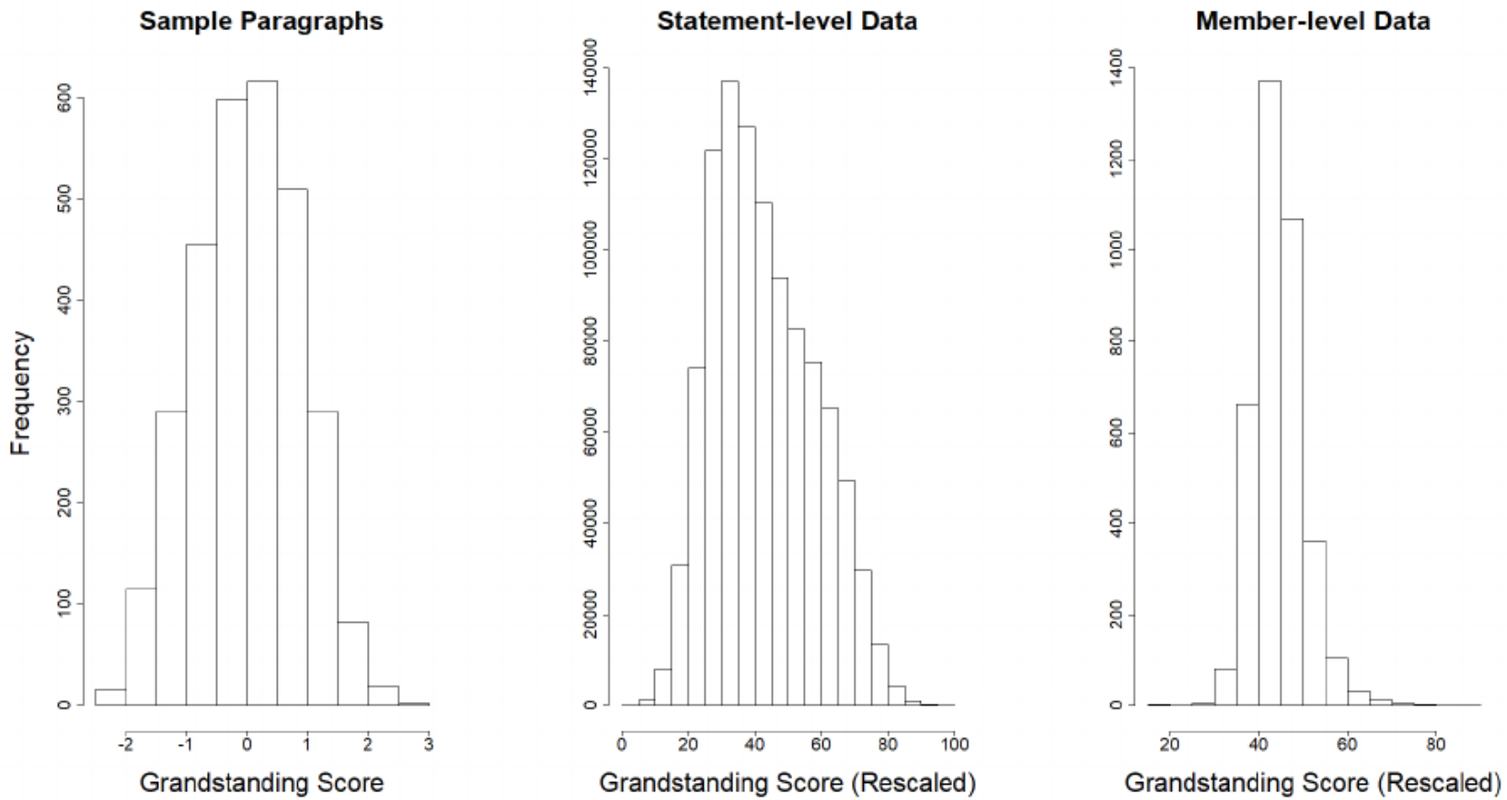
While congressional committee members sometimes hold hearings to collect and transmit specialized information to the floor, they also use hearings as venues to send political messages by framing an issue or a party to the public which I refer to as "grandstanding." However, we lack clear understanding of when they strategically engage in grandstanding. I argue that when committee members have limited legislative power they resort to making grandstanding speeches in hearings to please their target audience. Using 12,820 House committee hearing transcripts from the 105th to 114th Congresses and employing a crowd-sourced supervised learning method, I measure a "grandstanding score" for each statement that committee members make. Findings suggest that grandstanding efforts are made more commonly among minority members under a unified government, by nonchair members of powerful committees, and in committees with jurisdiction over policies where the president wields primary power, such as foreign affairs and national security.

- **Texts:** US House hearing statements.
- **Study:** crowd coding and grandstanding scores.
- **Our practice:** Park (2021) data, `rescaled_gs` labels; mlr3 linear SVM.

Park (2021), JOP 83(1): 214–228 · Author manuscript.

Grandstanding: predicting the score

Figure 1. Distribution of Grandstanding Score



Note: The grandstanding scores for the statements in the entire dataset are rescaled to run from 0 to 100 and are reflected in the last two graphs. The scores are aggregated by each member in each Congress in the member-level data.

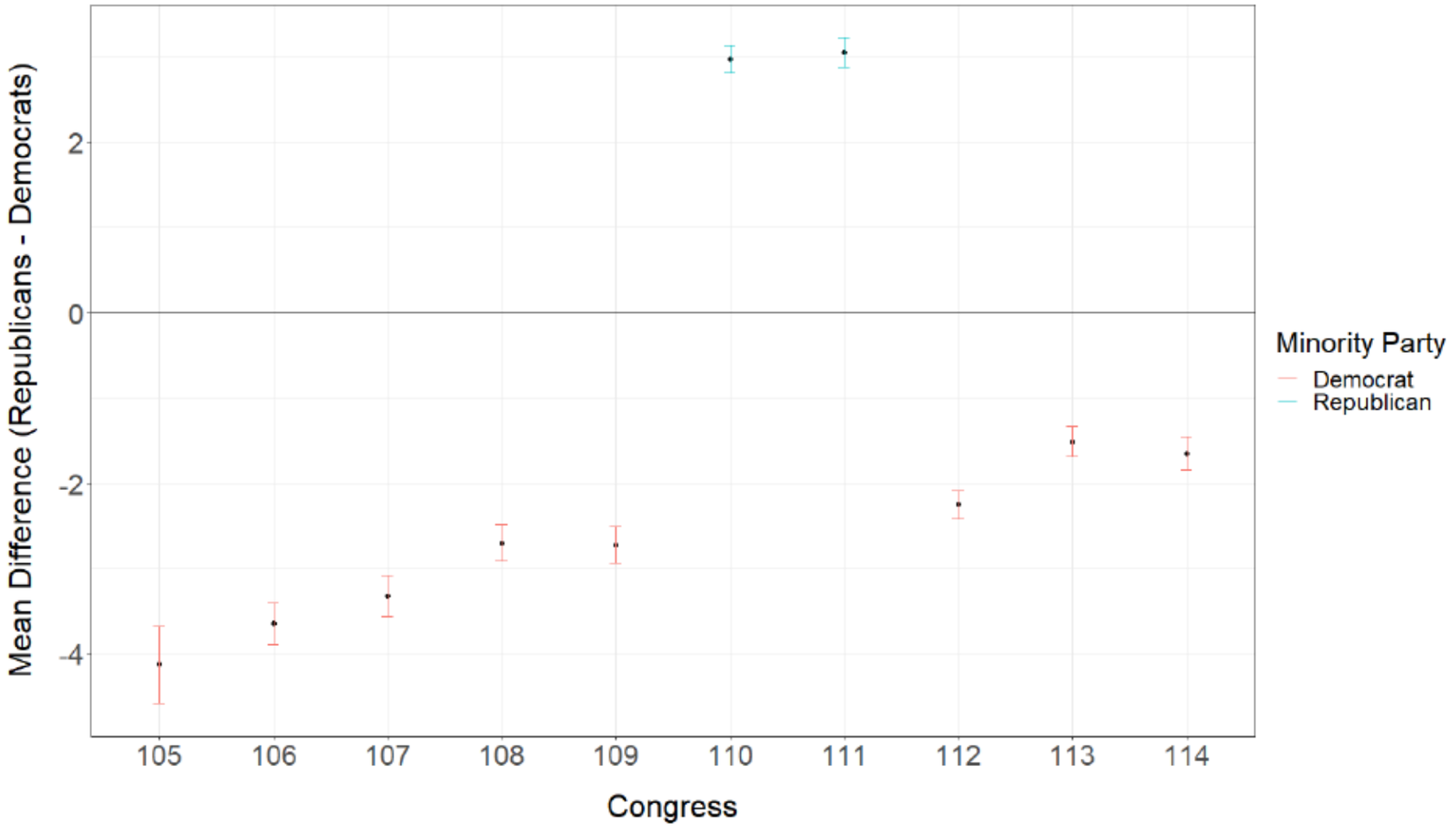
- **Labels:** 3,000 crowd-rated paragraphs.
- **Test:** 300 reserved paragraphs.
- **RMSE: 0.613**, original units.

Figure 1 · author manuscript

Park (2021), manuscript Figure 1 and pp. 16–18.

Grandstanding: using the measure in research

Figure 3. Partisan Difference in Grandstanding Scores Across Congresses



- **Axis:** Republican – Democratic means.
- **Pattern:** minority party scores higher.
- **Interpretation:** messaging and power.

Ju Yeon Park (2021)

Park (2021), manuscript Figure 3, p. 20.

Coding and measurement choices

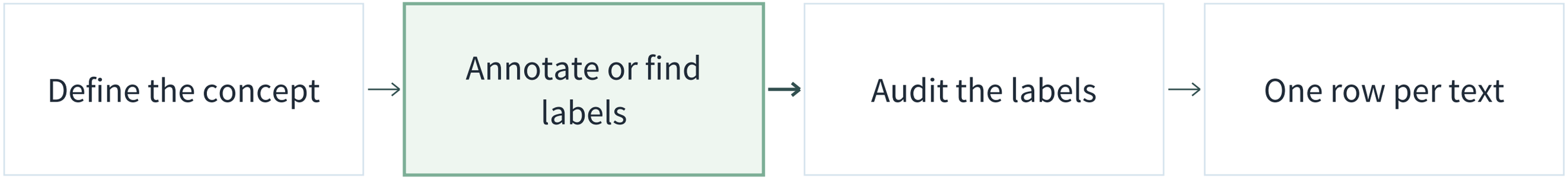
Course discussion: apply lessons from the research cases to our hate speech task.

Design choice	Question for W6
Coding unit	Does one sentence contain enough evidence for our category?
Repeated judgments	Which disagreements should trigger human review?
Aggregation	Do we need comment labels, hate-speech shares, or a genuinely numeric target?
Validation	Which independent evidence tests the intended interpretation?

More raters or models cannot repair an unsuitable coding definition.

Course synthesis from the preceding research cases, including Benoit et al. (2016).

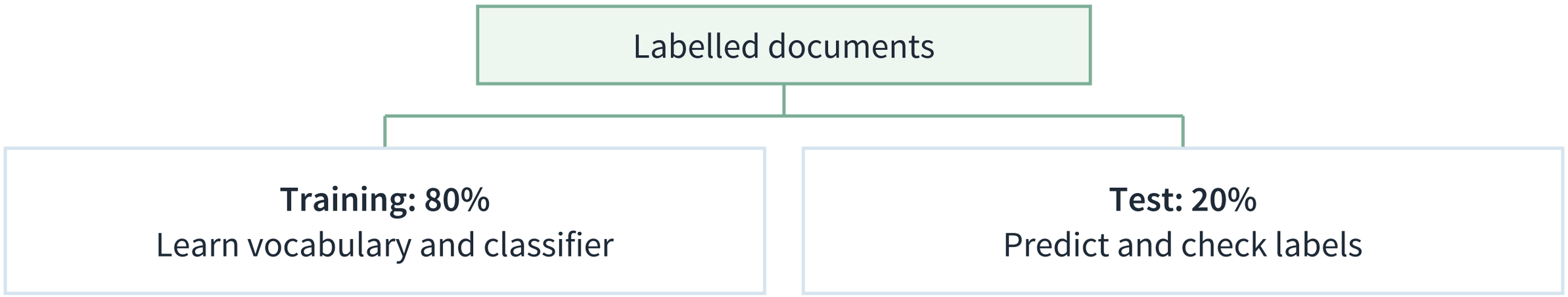
1. Obtain labelled data



- Create labels: write a codebook, collect independent judgments and adjudicate disagreements.
- Use existing labels: check the source, coding unit, definition and score-to-category conversion.
- Our cases: hate-speech labels from Wang, Day & Wu (2022) and the rescaled_gs categories in the Park (2021) data.

Data: Wang, Day & Wu (2022) · Park (2021) · Classroom ZIP.

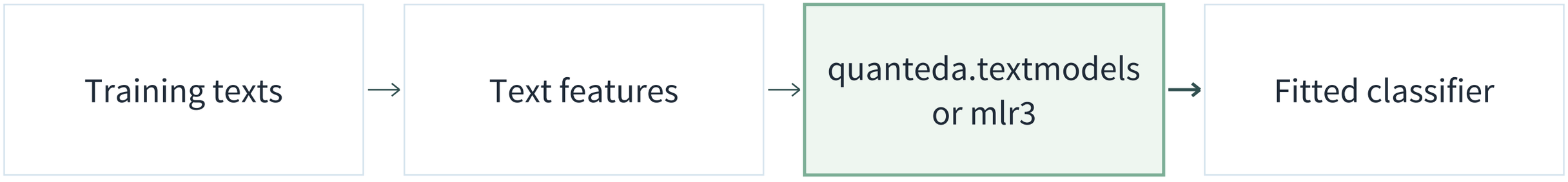
2. Split labelled documents



- Split the labelled documents before vocabulary selection or model fitting.
- Use a fixed stratified 80/20 split; keep test labels outside fitting.
- Apply the same workflow to both hate speech and grandstanding.

Data: Wang, Day & Wu (2022) · Park (2021) · Classroom ZIP.

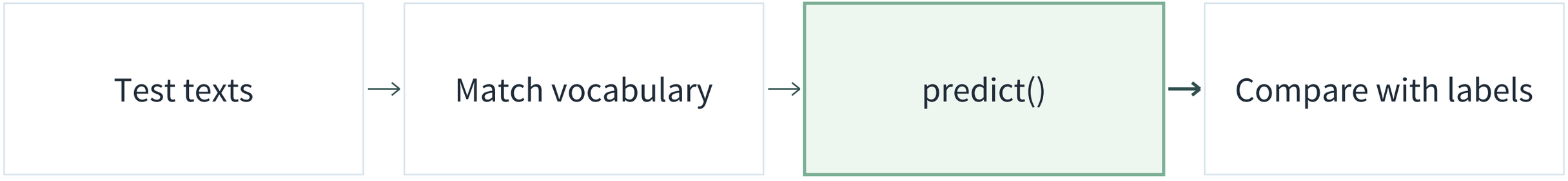
3. Fit ML Classifier



- Chinese comments: character counts. English excerpts: word counts.
- Choose the vocabulary using training texts only.
- Fit the classifier using the training feature matrix and matching labels.
- Classroom example: textmodel_nb() fits multinomial Naive Bayes.

Data: Wang, Day & Wu (2022) · Park (2021) · Classroom ZIP.

4. Predict and check labels



- Match test features to the training vocabulary, in the same order.
- Read predicted categories, the confusion table and F1.
- Hate-speech extension: check separate human-reference comments.

Data: Wang, Day & Wu (2022) · Park (2021) · Classroom ZIP.

R tools for document classification

R can prepare text features, train classifiers and evaluate predictions.

What we need	R tools
Convert text to features	`quanteda`
Fit a text classifier directly	`quanteda.textmodels`
Organize models and evaluation	`mlr3`
Connect preprocessing steps	`mlr3pipelines`
Search model settings	`mlr3tuning`

Choose mlr3 for resampling, model comparison, pipelines and tuning.

For direct fitting: use `quanteda.textmodels`, as in the W6 NB exercise.

Official documentation: quanteda · mlr3 · pipelines · tuning.

Introducing mlr3: machine learning in R

mlr3: Machine Learning in R

The mlr3 ecosystem is the framework for machine learning in R.

An open-source collection of R packages providing a unified interface for machine learning in the R language.
Successor of mlr.

A scientifically designed and easy to learn interface.

More than 100 connected machine learning algorithms.

Light on dependencies.

Convenient parallelization with the future package.

State-of-the-art optimization algorithms.

Dataflow programming with pipelines.

Official homepage · mlr-org.com · captured 2026-10-15

- **Framework:** one interface for different learners.
- **Task + learner:** define the target and fit a model.
- **Evaluation:** resampling, measures and extensions.

mlr3 official website · Applied Machine Learning Using mlr3 in R.

mlr3 and the Python ML ecosystem

Need	R: mlr3 ecosystem	Python: scikit-learn ecosystem
Model interface	Task + learner	Estimator
Fit / predict	`$train()` / `$predict()`	`.fit()` / `.predict()`
Text features	`quanteda` counts / TF-IDF	`CountVectorizer` / `TfidfVectorizer`
Preprocessing + model	`mlr3pipelines`	`Pipeline`
Cross-validation	`resample()`	`cross_validate()`
Parameter search	`mlr3tuning`	`GridSearchCV`

Same workflow: define the target, separate training and test data, then evaluate predictions.

mlr3 book · mlr3pipelines · mlr3tuning · scikit-learn guide · text features.

Simple models with quanteda.textmodels

For a simple text classifier, use a **DFM plus training labels** directly.

Model	Function
Naive Bayes	`textmodel_nb()`
Regularized logistic regression	`textmodel_lr()`
Linear SVM	`textmodel_svmlin()`

Linear SVM: experimental, for two classes.

W6: fit NB with training counts and labels, then predict held-out texts.

quanteda.textmodels manual · Package source and documentation.

ML Exercise (NB and SVM)

Case	Classifier	Model interface
Taiwan political hate speech	Multinomial Naive Bayes	`quanteda.textmodels`
Political grandstanding	Linear SVM	`mlr3` + `mlr3learners`

We will use the data from **Wang, Day & Wu (2022)** and **Park (2021)**.

Same operations: load → split → features → fit classifier → predict → check labels.

Wang, Day & Wu (2022); Park (2021) · Classroom ZIP.

Our datasets: Grandstanding and Hate Speech

Grandstanding Measurement (2021)

- **Unit:** English hearing excerpt.
- **Rows:** 3,500 source / 3,499 distinct.
- **Text → target:** `speech` → `rescaled_gs` (0/1).
- **Score field:** `sentimentit_score` (unused).

First 3 rows · `gscore_julia_park.csv`

Row	`speech` (preview)	`rescaled_gs`
1	Beautiful, because that brings me to my ne…	0
2	And so, while there may have been an indic…	1
3	Yeah, and I do not think there has been in…	0

Taiwan Political Hate Speech (2022)

- **Unit:** Chinese news comment.
- **Rows:** 31,071 source / 4,000 practice.
- **Text → target:** `text` → `hate_speech` (0/1).
- **Other field:** `label`, the category name.

First 3 rows · `merged-hate-speeches.csv`

Row	`text`	`hate_speech`
1	係阿6啊~	0
2	我媽媽說，學生最好騙，叫我不要上他的當	0
3	智X總統 畜X官員 無誤 ？！	1

Text previews follow the original CSV row order.

Park (2021); Wang, Day & Wu (2022).

R package check

```
# Install once
install.packages(c("quanteda", "quanteda.textmodels",
  "mlr3", "mlr3learners", "e1071", "stringi"),
  repos = "https://cloud.r-project.org")

# Load and check before the exercise
library(quanteda); library(quanteda.textmodels)
library(mlr3); library(mlr3learners)
library(e1071); library(stringi)
lrn("classif.svm")$id
```

Ready: all six packages load without errors; the last line returns `"classif.svm"`.

R: install.packages · mlr3 SVM learner.

The training data: what changes between cases?

Input	Hate speech	Grandstanding
One row	Chinese comment	English hearing excerpt
Label column	`hate_speech`	`rescaled_gs`
Positive category	HateSpeech	Grandstanding
Features	Chinese character counts	English word counts
Classifier	Multinomial NB	Linear SVM
Model interface	`quanteda.textmodels`	`mlr3` + `mlr3learners`

Only text enters the feature matrix. Labels are the outcome.

Data: Wang, Day & Wu (2022). W6 classroom NB run. Data: Park (2021). W6 classroom linear SVM run.

Labels come before the algorithm

- Read the coding definition and the unit being classified.
- Keep the provided labels; do not invent a new cutoff.
- Inspect disagreements and uncertain texts when interpreting errors.

Hate speech: use the labels in the Wang, Day & Wu (2022) data.

Grandstanding: use `rescaled_gs`; the CSV does not document its original binary cutoff.

Data: Wang, Day & Wu (2022). W6 classroom NB run. Data: Park (2021). W6 classroom linear SVM run.

Naive Bayes: enough to start using it

Task: predict a category from text counts.

- Fit with a **document–feature matrix** and one label per row.
- NB combines the counted features as evidence for each category.
- `predict()` returns categories; `type = "probability"` returns model probabilities.

Today's focus: **fit → predict → inspect**.

Set up once for both cases

Work in pairs (two students). Copy and run — **C01**.

```
# C01: Setup
# install.packages(c("quanteda", "quanteda.textmodels",
#                    "mlr3", "mlr3learners", "e1071", "stringi"))
library(quanteda); library(quanteda.textmodels)
library(mlr3)
quanteda_options(threads = 1)
data_dirs <- c("Data", "W6/lab/Data",
               "W6/classroom/Data", "../Data")
data_dir <- data_dirs[dir.exists(data_dirs)][1]
file.exists(file.path(data_dir, "hate_speech_practice.csv"))
measures <- msrs(c("classif.acc", "classif.precision",
                   "classif.recall", "classif.fbeta"))
```

Unzip the package and open R in its folder.

C01 output: setup check

R console output

```
> file.exists(file.path(data_dir, "hate_speech_practice.csv"))
[1] TRUE
```

Function	Purpose
`quanteda_options()`	Set the text-processing options for this session
`msrs()`	Create the accuracy, precision, recall and F1 measure objects

Discuss: What would FALSE tell us about the data path?

Hate speech: load the data

Copy and run — C02.

```
# C02: Load the Wang, Day & Wu (2022) data
hate <- read.csv(file.path(data_dir,
  "hate speech practice.csv"), stringsAsFactors = FALSE)
hate$label <- factor(hate$label,
  levels = c("NonHateSpeech", "HateSpeech"))
table(hate$label)
nrow(hate)
```

`read.csv()` loads 4,000 cleaned comments with the existing labels.

Data: Wang, Day & Wu (2022). W6 classroom NB run.

C02 output: class labels

R console output

```
> table(hate$label)
NonHateSpeech    HateSpeech
         1622          2378
> nrow(hate)
[1] 4000
```

Discuss: Which class is more common? What would always predicting that class achieve?

Data: Wang, Day & Wu (2022). W6 classroom NB run.

Hate speech: split documents

Copy and run — C03.

```
# C03: Reserve test comments with mlr3
hs_task <- as_task_classif(hate["label"],
  target = "label", positive = "HateSpeech")
hs_task$col_roles$stratum <- "label"
set.seed(7192)
hs_split <- partition(hs_task, ratio = .8)
parts <- list(train = hate[hs_split$train, ],
              test = hate[hs_split$test, ])
table(parts$train$label)
table(parts$test$label)
```

mlr3: `as_task_classif()` declares the outcome. `partition()` returns row IDs for the split.

Data: Wang, Day & Wu (2022). W6 classroom NB run.

C03 output: training and test counts

R console output

```
> table(parts$train$label)
NonHateSpeech    HateSpeech
         1298          1902
> table(parts$test$label)
NonHateSpeech    HateSpeech
          324           476
```

Discuss: Why do we reserve test rows before selecting the vocabulary?

Data: Wang, Day & Wu (2022). W6 classroom NB run.

Hate speech: tokenize Chinese characters

Copy and run — C04a_1.

```
# C04a_1: Normalize and tokenize Chinese comments
hs_text <- stringi::stri_trans_nfkc(hate$text)
names(hs_text) <- hate$doc_id
hs_tokens <- tokens(hs_text, what = "character",
  remove_punct = TRUE, remove_symbols = TRUE,
  remove_numbers = TRUE)
class(hs_tokens)
```

Function / object	What it does here
`stri_trans_nfkc()`	stringi: normalize Unicode forms in a working text copy.
`quanteda::tokens()`	Turn each comment into character tokens.

Data: Wang, Day & Wu (2022). W6 classroom NB run. · quanteda tokens

C04a_1 output: the tokens object

R console output

```
> class(hs_tokens)
[1] "tokens"
```

Discuss: Why use character tokens here, and what might we lose?

Data: Wang, Day & Wu (2022). W6 classroom NB run. · quanteda tokens

Hate speech: turn tokens into a DFM

Copy and run — C04a_2.

```
# C04a_2: Select Han characters and count features
hs_tokens <- tokens_select(hs_tokens,
  pattern = "^\\p{Han}$", valuetype = "regex")
hs_dfm <- dfm(hs_tokens)
dim(hs_dfm)
```

Function / object	What it does here
`tokens_select()`	Retain tokens matching the Han-character pattern.
`dfm()`	Count features in a sparse document-feature matrix.

Data: Wang, Day & Wu (2022). W6 classroom NB run. · tokens_select · dfm

C04a_2 output: character counts

R console output

```
> dim(hs_dfm)
[1] 4000 2916
```

Discuss: What do the 4,000 rows and 2,916 columns represent?

Data: Wang, Day & Wu (2022). W6 classroom NB run. · tokens_select · dfm

Hate speech: select the training vocabulary

Copy and run — C04b_1.

```
# C04b_1: Trim training features
train_dfm <- dfm_trim(hs_dfm[hs_split$train, ],
  min_docfreq = .01, docfreq_type = "prop")
train_label <- parts$train$label
dim(train_dfm)
```

Function / object	What it does here
`dfm_trim()`	Keep features occurring in at least 1% of training documents.
`train_label`	Keep one label per training row, in the same order.

Data: Wang, Day & Wu (2022). W6 classroom NB run. · dfm_trim

C04b_1 output: the training vocabulary

R console output

```
> dim(train_dfm)
[1] 3200  603
```

Discuss: Does min_docfreq count documents or total token occurrences?

Data: Wang, Day & Wu (2022). W6 classroom NB run. · dfm_trim

Hate speech: match the test columns

Copy and run — C04b_2.

```
# C04b_2: Match test features to training columns
test_dfm <- dfm_match(hs_dfm[hs_split$test, ],
  features = featnames(train_dfm))
dim(test_dfm)
identical(featnames(train_dfm), featnames(test_dfm))
```

Function / object	What it does here
`featnames()`	Read the training feature names in column order.
`dfm_match()`	Match that order, drop extra features and fill missing ones with zero.

Data: Wang, Day & Wu (2022). W6 classroom NB run. · dfm_match

C04b_2 output: matching test features

R console output

```
> dim(test_dfm)
[1] 800 603
```

```
> identical(featnames(train_dfm), featnames(test_dfm))
[1] TRUE
```

Discuss: Why must the 800 test rows use the same 603 columns in the same order?

Data: Wang, Day & Wu (2022). W6 classroom NB run. · dfm_match

Hate speech: fit NB

Copy and run — C05.

```
# C05: Fit Naive Bayes
nb <- textmodel_nb(train_dfm, y = train_label,
  distribution = "multinomial", smooth = 1,
  prior = "docfreq")
class(nb)
nb$distribution
```

quanteda.textmodels: `textmodel_nb()` takes counts and matching labels, then returns a fitted NB model.

C05 output: the fitted NB model

R console output

```
> class(nb)
[1] "textmodel_nb" "textmodel"    "list"
> nb$distribution
[1] "multinomial"
```

Argument	What we supply
`x`	Training feature counts
`y`	Labels in the same row order
`smooth = 1`	Smoothing for counts
`prior = "docfreq"`	Training label frequencies

Discuss: Which argument supplies the training labels?

Hate speech: predict and check

Copy and run — C06.

```
# C06: Predict and evaluate with mlr3
nb_prediction <- predict(nb, newdata = test_dfm)
nb_check <- PredictionClassif$new(task = hs_task,
  row_ids = hs_split$test,
  truth = hs_task$truth(hs_split$test),
  response = as.character(nb_prediction))
nb_check$confusion
nb_scores <- nb_check$score(measures)
round(nb_scores, 3)
```

Interfaces: `predict()` uses the quanteda NB model. `PredictionClassif$new()` combines predictions with test labels. `$score()` computes metrics.

Data: Wang, Day & Wu (2022). W6 classroom NB run.

C06 output: predictions and errors

R console output

```
> nb_check$confusion
                truth
response         HateSpeech NonHateSpeech
  HateSpeech             371            140
  NonHateSpeech          105            184
> round(nb_scores, 3)
     classif.acc classif.precision    classif.recall     classif.fbeta
           0.694             0.726             0.779             0.752
```

Discuss (2 minutes): Rows are predictions. Which cells are false positives and false negatives?

Data: Wang, Day & Wu (2022). W6 classroom NB run.

Hate speech: predict one actual comment

Copy and run — C07.

```
# C07: Inspect one held-out comment
i <- match("hs_26234", parts$test$doc_id)
cat(parts$test$text[i], "\n")
predict(nb, newdata = test_dfm[i, ])
round(predict(nb, newdata = test_dfm[i, ],
              type = "probability"), 3)
parts$test$label[i]
```

The full comment uses the **same training vocabulary**.

Data: Wang, Day & Wu (2022). W6 classroom NB run.

C07 output: one hate-speech prediction

垃圾426，滾回去中國，永不得進來台灣（You mainland Chinese scum, get back to China and never set foot in Taiwan.）

Comment ID: `hs_26234`

What to read	Result for this comment
Model prediction	`HateSpeech`
Model probability	`P(HateSpeech) = 0.999`
Reference label	`HateSpeech`

Discuss: Which words express group-based degradation or exclusion?

Data: Wang, Day & Wu (2022). W6 classroom NB run.

C07: inspect the R output

R console output

```
> predict(nb, newdata = test_dfm[i, ])
  hs_26234
HateSpeech
Levels: NonHateSpeech HateSpeech
> round(predict(nb, newdata = test_dfm[i, ], type = "probability"), 3)
docs        NonHateSpeech HateSpeech
  hs_26234         0.001      0.999
> parts$test$label[i]
[1] HateSpeech
Levels: NonHateSpeech HateSpeech
```

- `Levels` lists the two possible categories.
- `0.999` is a model probability, not overall accuracy.

Data: Wang, Day & Wu (2022). W6 classroom NB run.

A separate human-reference check

- **Merged file:** `hate_speech_reference.csv` combines two 1,000-comment validation files.
- **BERT / Lexicon:** names of the original files, recorded in the `source` column.
- **Reference label:** `label` comes from 是否仇恨 (**Hate speech: Yes/No**), not the original model-prediction columns.
- **1,968 distinct comments** remain after duplicate/conflict checks and are excluded from training.

We use these comments **after fitting**, with no model selection on them.

Data: Wang, Day & Wu (2022). W6 classroom NB run.

Hate speech: human-reference features

Copy and run — C08a.

```
# C08a: Match human-reference texts to the vocabulary
benchmark <- read.csv(file.path(data_dir,
  "hate_speech_reference.csv"), stringsAsFactors = FALSE)
benchmark$label <- factor(benchmark$label,
  levels = c("NonHateSpeech", "HateSpeech"))
ref_text <- stringi::stri_trans_nfkc(benchmark$text)
names(ref_text) <- benchmark$doc_id
ref_tokens <- tokens(ref_text, what = "character",
  remove_punct = TRUE, remove_symbols = TRUE,
  remove_numbers = TRUE)
ref_dfm <- dfm_match(dfm(ref_tokens),
  features = featnames(train_dfm))
dim(ref_dfm); sum(ntoken(ref_dfm) == 0)
```

Data: Wang, Day & Wu (2022). W6 classroom NB run.

C08a output: reference features

R console output

```
> dim(ref_dfm)
[1] 1968  603
> sum(ntoken(ref_dfm) == 0)
[1] 17
```

Discuss: Do the 17 zero-feature rows mean empty comments or no retained characters?

Data: Wang, Day & Wu (2022). W6 classroom NB run.

Hate speech: check human references

Copy and run — C08b.

```
# C08b: Evaluate the separate human references
ref_task <- as_task_classif(benchmark["label"],
  target = "label", positive = "HateSpeech")
benchmark$prediction <- predict(nb, newdata = ref_dfm)
ref_check <- PredictionClassif$new(task = ref_task,
  row_ids = ref_task$row_ids,
  response = as.character(benchmark$prediction))
benchmark_scores <- ref_check$score(measures)
nrow(benchmark); round(benchmark_scores, 3)
```

Evaluation: `PredictionClassif$new()` stores reference and prediction together. `$score(measures)` calculates the selected metrics.

Data: Wang, Day & Wu (2022). W6 classroom NB run.

C08b output: human-reference evaluation

R console output

```
> nrow(benchmark)
[1] 1968
> round(benchmark_scores, 3)
 classif.acc classif.precision    classif.recall     classif.fbeta
        0.514             0.165             0.880             0.278
```

Discuss (2 minutes): Recall is .880, but precision is .165. What errors does that imply?

Data: Wang, Day & Wu (2022). W6 classroom NB run.

Hate speech: compare evaluation samples

Check	Internal test	Human references
Comments	800	1,968
HateSpeech precision	.726	.165
HateSpeech recall	.779	.880
HateSpeech F1	.752	.278

Read: high recall alone is insufficient. This simple NB produces many false positives on the separate sample.

Data: Wang, Day & Wu (2022). W6 classroom NB run.

Grandstanding: source and lab columns

Source: `gscore_julia_park.csv`.

Lab: `grandstanding_practice.csv`.

- `speech` becomes `text`: English hearing excerpt.
- `rescaled_gs` becomes `label`: existing binary category.
- `sentimentit_score` stays outside the predictors.

Build English word counts with quanteda; fit a **linear SVM with mlr3**.

Data: Park (2021). W6 classroom linear SVM run.

Grandstanding: load the data

Copy and run — G01a.

```
# G01a: Load the Park (2021) data
grandstanding <- read.csv(file.path(data_dir,
  "grandstanding_practice.csv"), stringsAsFactors = FALSE)
grandstanding$label <- factor(grandstanding$label,
  levels = c("NonGrandstanding", "Grandstanding"))
table(grandstanding$label)
```

The CSV retains `rescaled_gs` categories; one duplicate was removed before class.

Data: Park (2021). W6 classroom linear SVM run.

G01a output: grandstanding labels

R console output

```
> table(grandstanding$label)
NonGrandstanding    Grandstanding
            1846             1653
```

Discuss: Which class is more common in the Park (2021) data?

Data: Park (2021). W6 classroom linear SVM run.

Grandstanding: reserve test excerpts

Copy and run — G01b.

```
# G01b: Reserve test excerpts with mlr3
gs_task <- as_task_classif(grandstanding["label"],
  target = "label", positive = "Grandstanding")
gs_task$col_roles$stratum <- "label"
set.seed(7192)
gs_split <- partition(gs_task, ratio = .8)
gs_parts <- list(train = grandstanding[gs_split$train, ],
                 test = grandstanding[gs_split$test, ])
table(gs_parts$train$label)
table(gs_parts$test$label)
```

mlr3: `partition()` returns row IDs. Setting `stratum` to the label preserves an approximate 80/20 split within each category.

Data: Park (2021). W6 classroom linear SVM run.

G01b output: training and test counts

R console output

```
> table(gs_parts$train$label)
NonGrandstanding    Grandstanding
            1477             1322
> table(gs_parts$test$label)
NonGrandstanding    Grandstanding
             369              331
```

Discuss: How does stratification preserve both categories in the test set?

Data: Park (2021). W6 classroom linear SVM run.

Grandstanding: build a corpus and tokens

Copy and run — G02a_1.

```
# G02a_1: Build an English corpus and tokens
gs_corpus <- corpus(grandstanding,
  text_field = "text", docid_field = "doc_id")
gs_tokens <- tokens(gs_corpus, remove_punct = TRUE,
  remove_symbols = TRUE, remove_numbers = TRUE,
  remove_url = TRUE)
ndoc(gs_corpus)
```

Function / object	What it does here
`corpus()`	Read text and document IDs from the data table.
`tokens()`	Split the selected text into words and remove the requested token types.

Data: Park (2021). W6 classroom linear SVM run. · corpus · tokens

G02a_1 output: the English corpus

R console output

```
> ndoc(gs_corpus)
[1] 3499
```

Discuss: Which column supplies text, and which column supplies document IDs?

Data: Park (2021). W6 classroom linear SVM run. · corpus · tokens

Grandstanding: normalize and count words

Copy and run — G02a_2.

```
# G02a_2: Normalize English tokens and build a DFM
gs_tokens <- tokens_tolower(gs_tokens)
gs_tokens <- tokens_remove(gs_tokens,
  pattern = stopwords("en"))
gs_dfm <- dfm(gs_tokens)
dim(gs_dfm)
```

Function / object	What it does here
`tokens_tolower()`	Combine upper- and lowercase versions of words.
`tokens_remove()`	Remove the English stopword list from the tokens.
`dfm()`	Count the remaining words for each document.

Data: Park (2021). W6 classroom linear SVM run. · tokens_tolower · tokens_remove · dfm

G02a_2 output: English word counts

R console output

```
> dim(gs_dfm)
[1]  3499 14123
```

Discuss: How do these word features differ from the Chinese character features?

Data: Park (2021). W6 classroom linear SVM run. · tokens_tolower · tokens_remove · dfm

Grandstanding: trim and align the vocabulary

Copy and run — G02a_3.

```
# G02a_3: Select training words and match test columns
gs_train_dfm <- dfm_trim(gs_dfm[gs_split$train, ],
  min_docfreq = .01, docfreq_type = "prop")
gs_test_dfm <- dfm_match(gs_dfm[gs_split$test, ],
  features = featnames(gs_train_dfm))
dim(gs_train_dfm); dim(gs_test_dfm)
```

Function / object	What it does here
`dfm_trim()`	Choose words using training document frequencies.
`dfm_match()`	Give test documents the same feature columns.

Data: Park (2021). W6 classroom linear SVM run. · dfm_trim · dfm_match

G02a_3 output: aligned English features

R console output

```
> dim(gs_train_dfm)
[1] 2799  629
> dim(gs_test_dfm)
[1] 700 629
```

Discuss: Why do both matrices have 629 columns even though they contain different texts?

Data: Park (2021). W6 classroom linear SVM run. · dfm_trim · dfm_match

Grandstanding: convert the DFM for mlr3

Copy and run — G02b_1.

```
# G02b_1: Convert text counts to feature tables
gs_train_data <- convert(gs_train_dfm, to = "data.frame")
gs_test_data <- convert(gs_test_dfm, to = "data.frame")
gs_train_data$doc_id <- NULL; gs_test_data$doc_id <- NULL
dim(gs_train_data); dim(gs_test_data)
```

Function / object	What it does here
`quanteda::convert()`	Turn the small sparse DFM into a dense data frame.
`doc_id <- NULL`	Remove the identifier, leaving 629 numeric feature columns.

Data: Park (2021). W6 classroom linear SVM run. · quanteda convert

G02b_1 output: feature tables without IDs

R console output

```
> dim(gs_train_data)
[1] 2799  629
> dim(gs_test_data)
[1] 700 629
```

Discuss: Why should a document ID stay outside the predictor columns?

Data: Park (2021). W6 classroom linear SVM run. · quanteda convert

Grandstanding: add the target column

Copy and run — G02b_2.

```
# G02b_2: Name feature columns and add labels
names(gs_train_data) <- make.names(featnames(gs_train_dfm),
  unique = TRUE)
names(gs_test_data) <- names(gs_train_data)
gs_train_data$label <- gs_parts$train$label
gs_test_data$label <- factor(gs_parts$test$label,
  levels = c("Grandstanding", "NonGrandstanding"))
dim(gs_train_data); dim(gs_test_data)
```

Function / object	What it does here
`make.names()`	Create valid, unique feature names.
`label`	Add the target after the 629 feature columns.

Data: Park (2021). W6 classroom linear SVM run. · mlr3 PredictionClassif

G02b_2 output: features plus the target

R console output

```
> dim(gs_train_data)
[1] 2799  630
> dim(gs_test_data)
[1] 700 630
```

630 columns = 629 text features + 1 `label` column.

- **Predictors:** the 629 word-count features.
- **Target:** `label` (Grandstanding / NonGrandstanding), excluded from predictors.

Discuss: Why are there now 630 columns, but only 629 predictors?

Data: Park (2021). W6 classroom linear SVM run. · mlr3 PredictionClassif

mlr3: define the classification task

Copy and run — G02c_1.

```
# G02c_1: Define the grandstanding task
gs_model_task <- as_task_classif(gs_train_data,
  target = "label", positive = "Grandstanding")
gs_model_task$target_names
length(gs_model_task$feature_names)
```

Function / object	What it does here
`as_task_classif()`	Describe the training data, target column and positive class.
`TaskClassif`	Stores data and roles. Creating the task does not train a model.

Data: Park (2021). W6 classroom linear SVM run. · as_task_classif

G02c_1 output: the classification task

R console output

```
> gs_model_task$target_names
[1] "label"
> length(gs_model_task$feature_names)
[1] 629
```

Discuss: Which column is the target, and why is it excluded from feature_names?

Data: Park (2021). W6 classroom linear SVM run. · as_task_classif

mlr3: choose an SVM learner

Copy and run — G02c_2.

```
# G02c_2: Configure the linear SVM learner
library(mlr3learners)
gs_learner <- lrn("classif.svm", type = "C-classification",
  kernel = "linear", cost = 1, scale = FALSE,
  predict type = "prob")
gs_learner$id
gs_learner$predict_type
```

Function / object	What it does here
`lrn()`	Create an unfitted learner and set its options.
`predict_type = "prob"`	Request class probabilities as well as predicted categories.

Data: Park (2021). W6 classroom linear SVM run. · mlr3 SVM

G02c_2 output: the configured learner

R console output

```
> gs_learner$id
[1] "classif.svm"
> gs_learner$predict_type
[1] "prob"
```

Discuss: Has lrn() already learned from any training rows?

Data: Park (2021). W6 classroom linear SVM run. · mlr3 SVM

mlr3: train and predict

Copy and run — G02c_3.

```
# G02c_3: Train the learner and predict held-out texts
set.seed(7193)
gs_learner$train(gs_model_task)
gs_result <- gs_learner$predict_newdata(gs_test_data)
gs_prediction <- gs_result$response
length(gs_prediction)
colnames(gs_result$prob)
```

Function / object	What it does here
`$train(task)`	Fit the learner using the training task.
`$predict_newdata()`	Return predictions for the held-out feature table.
`$response` / `$prob`	Read category labels / the matrix of class probabilities.

Data: Park (2021). W6 classroom linear SVM run. · mlr3 Learner · PredictionClassif

G02c_3 output: held-out predictions

R console output

```
> length(gs_prediction)
[1] 700
> colnames(gs_result$prob)
[1] "Grandstanding"    "NonGrandstanding"
```

Discuss: Which object stores the fitted learner, and which stores the 700 test predictions?

Grandstanding: evaluate mlr3 predictions

Copy and run — G02d.

```
# G02d: Evaluate the mlr3 predictions
gs_result$confusion
gs_scores <- gs_result$score(measures)
round(gs_scores, 3)
```

mlr3 fields and methods: `$confusion` reads a table. `$score(measures)` calls a method that returns evaluation scores.

Data: Park (2021). W6 classroom linear SVM run.

G02d output: predictions and errors

R console output

```
> gs_result$confusion
                truth
response         Grandstanding NonGrandstanding
  Grandstanding              176                74
  NonGrandstanding           155               295
> round(gs_scores, 3)
     classif.acc classif.precision    classif.recall     classif.fbeta
           0.673             0.704             0.532             0.606
```

Discuss: Locate the 74 false positives and 155 false negatives.

Data: Park (2021). W6 classroom linear SVM run.

Grandstanding: predict one actual excerpt

Copy and run — G03.

```
# G03: Inspect one grandstanding prediction
j <- match("gs_01045", gs_parts$test$doc_id)
cat(gs_parts$test$text[j], "\n")
gs_result$response[j]
round(gs_result$prob[j, ], 3)
gs_result$truth[j]
```

The full excerpt is held outside training.

Data: Park (2021). W6 classroom linear SVM run.

G03 output: one grandstanding prediction

R console output

```
> gs_result$response[j]
[1] Grandstanding
Levels: Grandstanding NonGrandstanding
> round(gs_result$prob[j, ], 3)
   Grandstanding NonGrandstanding
           0.637             0.363
> gs_result$truth[j]
[1] Grandstanding
Levels: Grandstanding NonGrandstanding
```

Discuss: Which words suggest self-promotion? Read the full statement next.

Data: Park (2021). W6 classroom linear SVM run.

Grandstanding: read the high-score statement

Also I have not signed the no tax pledge. ***I am not intimidated by anybody. I am going to do whatever I have to do.*** I do not like to raise taxes. I do not want to raise taxes. I want to see how we can cut first. But, I also want to deal with the entitlements. The Simpson Bowles Commission is sort of a grand compromise. A grand compromise that not everyone really feels totally comfortable with. But you know that they are kind of moving in the right direction. So we really are going to have to get it out of this administration to participate with us.

Prediction / reference: Grandstanding · **SVM probability:** .637

Read: self-promotion and a public stance on taxes. Original score: **2.003**.

Data: Park (2021). W6 classroom linear SVM run.

Read performance with the text

Run after G03 in the same R session.

```
# Read one grandstanding prediction error
error_rows <- which(gs_result$response != gs_result$truth)
k <- error_rows[1]
data.frame(
  doc_id = gs_parts$test$doc_id[k],
  reference = gs_result$truth[k],
  prediction = gs_result$response[k],
  prob = round(gs_result$prob[k, "Grandstanding"], 3),
  row.names = NULL)
cat(gs_parts$test$text[k], "\n")
```

Read: `which()` finds mismatches, `k` selects one, and `cat()` prints its full text.

Data: Park (2021). W6 classroom linear SVM run.

Read performance: one false positive

R console output

```
    doc_id         reference    prediction  prob
1 gs_00013 NonGrandstanding Grandstanding 0.609
```

I know that Ms. Pingree mentioned the Coastal Climate Resilience Fund. So the district I represent includes a whole bunch of rural communities that lie in the tsunami zone. Coastal communities like Westport and Ocean Shores are seeing much more severe storm events and flooding and there are four coastal tribes in the district that I represent that are in the process of trying to move to higher ground. I was hoping you could provide a roadmap for what this program is going to look like if it is funded, and how the program is going to address the specific needs of the communities I represent and others along the coast.

Discuss: Does describing district needs count as self-promotion? What context is missing?

Data: Park (2021). W6 classroom linear SVM run.

Optional: reuse the features with other classifiers

Use the same training split and vocabulary.

Model	Package interface	Prediction
Logistic regression	`quanteda.textmodels::textmodel_lr()`	`predict()`
Linear SVM	mlr3 `lrn("classif.svm", kernel = "linear")`	`$train()` then `$predict()`

The lab shows the complete calls. No custom model wrappers are required.

quanteda.textmodels · mlr3 classif.svm.

A short methods report

Report five things:

1. **Data and labels:** which text column and supplied target?
2. **Split:** mlr3 `partition()`, stratified 80/20, seed 7192.
3. **Features:** Chinese characters or English words; training vocabulary only.
4. **Model:** quanteda multinomial NB or mlr3 linear SVM; report its settings.
5. **Results:** test metrics and one error you inspected.

Data: Wang, Day & Wu (2022). W6 classroom NB run. Data: Park (2021). W6 classroom linear SVM run.

Later weeks: LLM coding and classification



ARTICLE | Open Access

Using large language models to analyze political texts through natural language understanding

Kenneth Benoit, Scott De Marchi, Conor Laver, Michael Laver, Jinshuai Ma

First published: 19 March 2026 | https://doi.org/10.1111/ajps.70050 | VIEW METRICS

Find It@NTU

Verification Materials: The materials required to verify the computational reproducibility of the results, procedures and analyses in this article are available on the *American Journal of Political Science* Dataverse within the Harvard Dataverse Network at: **https://doi.org/10.7910/DVN/XY1FFE**.

The Cornell Center for Social Sciences verified that the data and replication code submitted to the *AJPS* Dataverse replicate the numerical results reported in the main text of this article.

- **Classification:** prompts and codebooks can guide category labels.
- **AJPS example:** summaries and policy-position scores.
- **Later LLM weeks:** methods and validation.

Benoit et al. (2026), AJPS.

Practice and the Week 7 deadline

In class: copy **C01–C08**, then **G01–G03** (including all substeps), in order in one R session.

- Hate speech: inspect the test and separate human-reference results.
- Grandstanding: fit mlr3 linear SVM using `rescaled_gs`.
- Read one error from each case and explain the coding boundary.

Classroom code + data · Lab · Worked answers

Week 7: proposal due **2026-10-23, before 23:59**. No separate submission for this practice.

Recommended reading priorities

Text as Data chapter	Sections to prioritize	Teaching connection
17: An Overview of Supervised Classification	Overview: 17.2–17.5	Connect the research steps
18: Coding a Training Set	**Core: 18.1–18.2**	Codebooks and human labels
19: Classifying Documents with Supervised Learning	Model detail: 19.1–19.2	NB and learned classifiers
20: Checking Performance	**Core: 20.1**	Validation and prediction errors

Extensions: §19.2.3 for aggregate measures; §20.2 for indirect validation.

Chapter 15 provides optional measurement context.

Grimmer, Roberts & Stewart (2022), Text as Data, Part IV.

Practice data references

Wang, Chih-Chien, Min-Yuh Day, and Chun-Lian Wu. 2022. “Political Hate Speech Detection and Lexicon Building: A Study in Taiwan.” *IEEE Access* 10: 44337–44346. DOI.

Hate-speech data: Wang, Day & Wu (2022), `merged-hate-speeches.csv`.

Grandstanding data: Park (2021), `gscore_julia_park.csv`, target `rescaled_gs`.

Manual benchmark: the supplied BERT and Lexicon 1,000-comment validation sheets.

W6 results are new classroom runs, separate from the paper’s BERT and lexicon results.

Source files and hashes: data notes.

Empirical study references

- **Gillion, Daniel Q. 2016.** *Governing with Words: The Political Dialogue on Race, Public Policy, and Inequality in America*. Cambridge University Press. Appendix A.
- **Boydstun, Amber E. 2013.** *Making the News: Politics, the Media, and Agenda Setting*. University of Chicago Press. Codebook, data, and tables.
- **Nielsen, Richard A. 2017.** *Deadly Clerics: Blocked Ambition and the Paths to Jihad*. Cambridge University Press. Author's book page.

Methods supplements: Nielsen (2014); English & Nielsen (2024).

Additional coding study references

Park, Ju Yeon. 2021. “When Do Politicians Grandstand? Measuring Message Politics in Committee Hearings.” *The Journal of Politics* 83(1): 214–228. Article · Author manuscript.

Benoit, Kenneth, Drew Conway, Benjamin E. Lauderdale, Michael Laver, and Slava Mikhaylov. 2016. “Crowd-sourced Text Analysis: Reproducible and Agile Production of Political Data.” *American Political Science Review* 110(2): 278–295. Article · Replication.

Benoit, Kenneth, Scott De Marchi, Conor Laver, Michael Laver, and Jinshuai Ma. 2026. “Using large language models to analyze political texts through natural language understanding.” *American Journal of Political Science* 70(4): 1589–1605. Article · Replication.

References and software

Grimmer, J., M. E. Roberts, and B. M. Stewart (2022). *Text as Data: A New Framework for Machine Learning and the Social Sciences*. Princeton University Press. Chapters 17–20, with Chapter 15 as background. Book and contents.

Software: quanteda for text features, `quanteda.textmodels::textmodel_nb()` for Naive Bayes, `quanteda.textmodels::textmodel_lr()` for optional logistic regression, and mlr3 `classif.svm` (with `mlr3learners` and `e1071`) for the grandstanding linear SVM.

Further tools: quanteda.textmodels for text classifiers; mlr3 for splitting and evaluation; Transformers for the fine-tuning extension.

R workflow references: tidyverse, tidymodels, and the mlr3 feature-selection chapter.

Evaluation references: scikit-learn documentation on cross-validation and classification metrics.

Hate-speech NB uses Wang, Day & Wu (2022); grandstanding linear SVM uses Park (2021). Both have executed predictions. Named study results retain their original citations.